Reviewer Pack — Asymptotic-dimension lower bound for the indexing function v…

Entry e871be18b26c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 10:37:59 UTC

Asymptotic-dimension lower bound for the indexing function via controlled covers matches its log-depth

Entry ID: `e871be18b26c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-27 10:37:59 UTC

1. Conjecture

Field A (mathematical branch): Coarse Geometric Karchmer-Wigderson (CG-KW); coarse geometry / Roe algebras / metric (Roe) cohomology **Field B** (complexity object): Formula depth (over the De Morgan basis, equivalently NC^1 circuit depth) of the addressing/indexing function IND_k : {0,1}^{k + 2^k} -> {0,1}, which selects data bit x_a where a is encoded by the k address bits; IND_k has known formula depth Θ(k) and is the canonical KW-easy test instance for any proposed depth measure.

Statement:

> For every k ≥ 1, the KW-metric space (X_{IND_k}, d_{IND_k}) admits no controlled cover of diameter R = 2^{k-1} − 1 with multiplicity m ≤ k, and consequently asdim(X_{IND_k}) ≥ k − O(1); together with axiom A1 this yields κ(IND_k) ≥ k − O(1) = Ω(log N) where N = k + 2^k is the input length, matching the true depth Θ(k) up to a constant. Equivalently: there exists an explicit coarse 1-cocycle c_k ∈ HX^1(X_{IND_k}) and a Roe-controlled operator T_k of propagation 2^{k} with |⟨c_k, T_k⟩| ≥ 2^{Ω(k)}, certifying a super-constant κ on this family.

Rationale (proposer's reasoning):

> This sub-conjecture stress-tests axiom A1 (Coarse-KW link) on a function whose true depth is precisely known: if A1 holds, the controlled-cover multiplicity must witness the Karchmer-Wigderson protocol's depth, so on IND_k we MUST recover k − O(1). It also exercises axiom A5 (Roe-index rigidity) by demanding an explicit cocycle/operator pair realizing the trace pairing — IND_k is the simplest function whose KW-tree is a complete binary tree, so the address-bit projections give natural candidates for c_k. Failure on IND_k would falsify A1; success calibrates the constants in A1 before attempting the Andreev / element-distinctness Ω(log^2 n) targets.

Taxonomy category: `KARCHMER_WIGDERSON` (status at proposal time:)

Framework membership: framework `fw_85a254b4a0`, role: elaboration

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `aeec1991215f005b`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For k∈{2,3,4}: empirical asdim slope (log m(R) vs log R, R∈{1,..,2^k}) ≥ k−2 AND trace-pairing ratio log|⟨c_k,T_k⟩|/log(propagation) ≥ 0.5·k. Baseline: across 20 random f on N=10, mean |trace ratio| ≤ 0.1 with max ≤ 0.25. All three conditions must hold simultaneously to count as SUPPORTED.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: `NOVEL` against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `Karchmer-Wigderson asymptotic dimension formula depth lower bound` - `coarse geometry Roe algebra circuit complexity communication` - `asymptotic dimension multiplicity cover indexing function NC1 depth`

Top relevant hits considered: - [http://arxiv.org/abs/2107.05128v2] Karchmer-Wigderson Games for Hazard-free Computation - [http://arxiv.org/abs/2002.07444v1] Multiparty Karchmer-Wigderson Games and Threshold Circuits - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1904.05483v2] Parallels Between Phase Transitions and Circuit Complexity? - [http://arxiv.org/abs/2312.08907v2] Roe algebras of coarse spaces via coarse geometric modules - [http://arxiv.org/abs/2408.07701v2] A categorical interpretation of Morita equivalence for dynamical von Neumann algebras - [http://arxiv.org/abs/2405.12883v2] Asymptotic analysis at any order of Helmholtz's problem in a corner with a thin layer: an algebraic approach - [http://arxiv.org/abs/1208.1981v3] General notions of depth for functional data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from collections import defaultdict

def log2_floor(n):
    if n <= 0:
        raise ValueError("log2_floor is not defined for non-positive numbers")
    return int(math.log2(n))

def controlled_cover(R, k):
    N = k + 2**k
```

```
    X = [(i >> k) & ((1 << k) - 1), (i & ((1 << k) - 1)) for i in range(N)]
    cover = defaultdict(int)

    def bfs(node, radius):
        queue = [node]
        visited = set()
        while queue:
            node = queue.pop(0)
            if node not in visited:
                visited.add(node)
                for neighbor in X:
                    if abs(node - neighbor[0]) <= radius and abs(neighbor[1] - node) <= radius:
                        cover[node] += 1
                        queue.append(neighbor)

    for i in range(N):
        bfs(i, R // 2)

    return cover

def trace_product(cocycle, operator):
    N = len(operator)
    result = 0
    for i in range(N):
        for j in range(N):
            if operator[i][j] != 0:
                result += cocycle[(i >> k) & ((1 << k) - 1), (i & ((1 << k) - 1))] * \
                          cocycle[(j >> k) & ((1 << k) - 1), (j & ((1 << k) - 1))]
    return result

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for k in [2, 3, 4]:
        N = k + 2**k
        X = [(i >> k) & ((1 << k) - 1), (i & ((1 << k) - 1)) for i in range(N)]

        def d_IND_k(x, y):
            return log2_floor(min(abs(x[0] - y[0]), abs(x[1] - y[1])))

        max_R = 2**k
        cover_multiplicity = [0] * (max_R + 1)

        for R in range(1, max_R + 1):
            cover = controlled_cover(R, k)
            multiplicity = sum(cover.values())
            cover_multiplicity[R] = multiplicity

        asdim_slope = (math.log2(sum(multiplicity ** 0.5 for multiplicity in cover_multiplicity)) /
                       math.log2(max_R + 1)) / math.log2(max_R + 1)

        T_k = [[0] * N for _ in range(N)]
        c_k = defaultdict(int)

        for i in range(N):
```

```
            for j in range(N):
                if (i >> k) & ((1 << k) - 1) != (j >> k) & ((1 << k) - 1):
                    T_k[i][j] = 1
                c_k[(i >> k) & ((1 << k) - 1), (i & ((1 << k) - 1))] = (-1) ** i

        trace_cocycle_operator = trace_product(c_k, T_k)

        results.append({
            "k": k,
            "asdim_slope": asdim_slope,
            "trace_ratio": abs(trace_cocycle_operator / (2**k))
        })

    mean_asdim_slope = sum(result["asdim_slope"] for result in results) / len(results)
    mean_trace_ratio = sum(result["trace_ratio"] for result in results) / len(results)

    conjecture_holds = all(result["asdim_slope"] >= k - 2 and result["trace_ratio"] >= 0.5 * k for

    return {
        "metric_name": "asdim_slope",
        "metric_value": mean_asdim_slope,
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else f"asdim_slope < k-2 or trace_ratio < 0.5*k f
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f'TRIAL: {result}')
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b843142.py", line 185, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b843142.py", line 143, in run_trial
    cover = controlled_cover(R, k)
            ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b843142.py", line 41, in controlled_co
    for i in range(1 << (log2_floor(node) + 1)):
                         ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9b843142.py", line 25, in log2_floor
    return math.floor(math.log2(n))
                      ^^^^^^^^^^^^
ValueError: math domain error
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with a ValueError (math domain error in log2_floor due to a non-positive argument) before producing any slope, trace-pairing, or baseline data, so none of the pre-registered support conditions can be evaluated. | next: Guard log2_floor against n ≤ 0 (e.g., handle the root/zero node explicitly) and rerun the controlled-cover and trace-pairing measurements for k ∈ {2,3,4} with the 20-seed random baseline.

11. Audit log (LLM calls)

Total LLM calls: 6

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	preregistration	claude_max	opus	0	0	6954
2	novelty	claude_max	opus	0	0	3443
3	novelty	claude_max	opus	0	0	9625
4	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20846
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11436
6	judge	claude_max	opus	0	0	7842

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 60145 ms total latency. Provider mix: {'claude_max': 4, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/e871be18b26c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e871be18b26c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e871be18b26c.tar.gz` (if generated)